TECHNICAL UNIVERSITY OF CLUJ-NAPOCA ROMANIA

EUROPEAN UNIVERSITY OF TECHNOLOGY

Faculty of Electronics, Telecommunications and Information Technology

Isolation Forest Anomaly Detection and Long Short-Term Memory Traffic Prediction

Ștefan-Darian Cîrnaț, Virgil Dobrotă

PROJECT OVERVIEW

- Real-time network monitoring using open-source tools
- Docker-based emulated environment in GNS3
- Prometheus & Grafana for metric collection and visualization
- AI modules for anomaly detection and traffic forecasting
- Full automation and real traffic generation via containers

WHY THIS PROJECT?

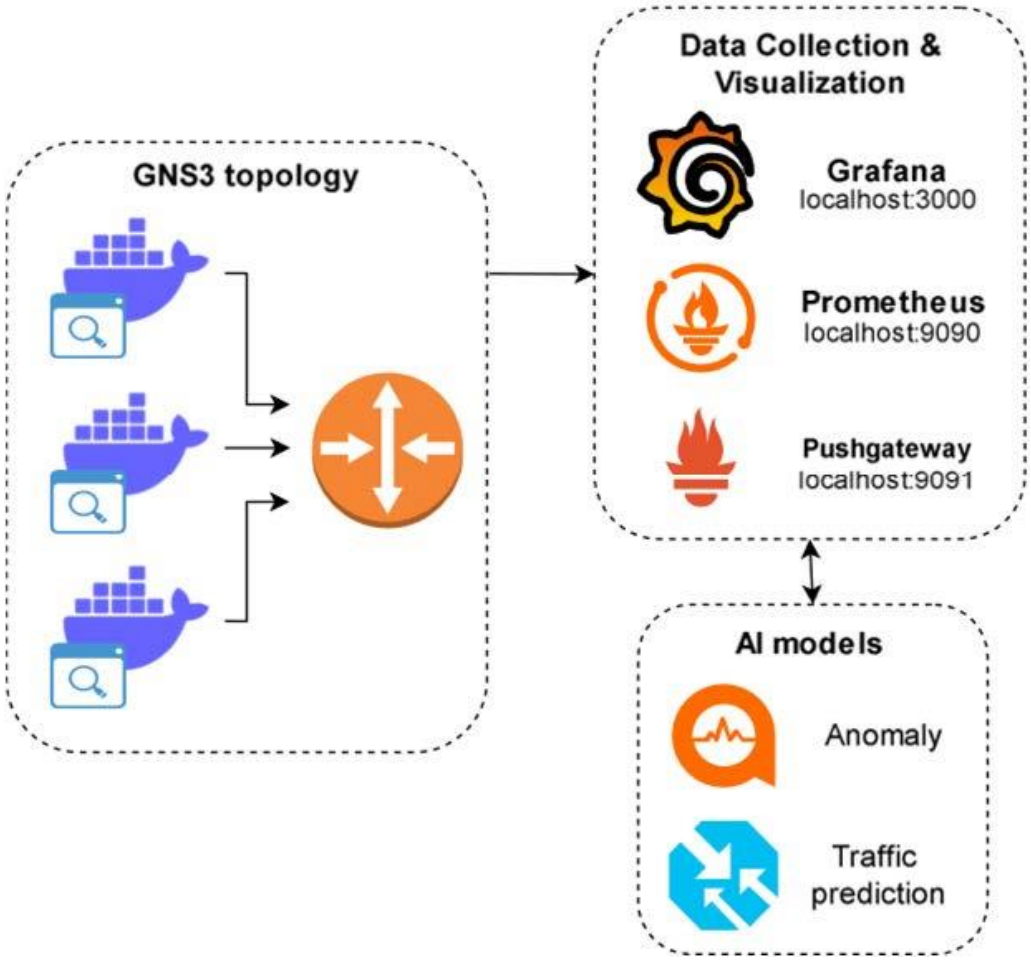
- Traditional monitoring systems:
  - often rely on static thresholds and reactive alerts
  - lack prediction capabilities and real anomaly detection
- Full enterprise legacy tools: Splunk, Cisco DNA Center. Disadvantages:
  - closed-source
  - expensive
  - complex for academic use
- Proposed solution: entirely from scratch, based on:
  - GNS3
  - Docker containers
  - Prometheus and Grafana
  - Python + ML models: Isolation Forest and Long Short-Term Memory Traffic
- Goals:
  - scalable, modular, and intelligent monitoring system
  - easy to deploy and to understand

SYSTEM ARCHITECTURE

GNS3 Topology:
- containers that generate traffic and send it through a router, enabling Prometheus to collect the metrics

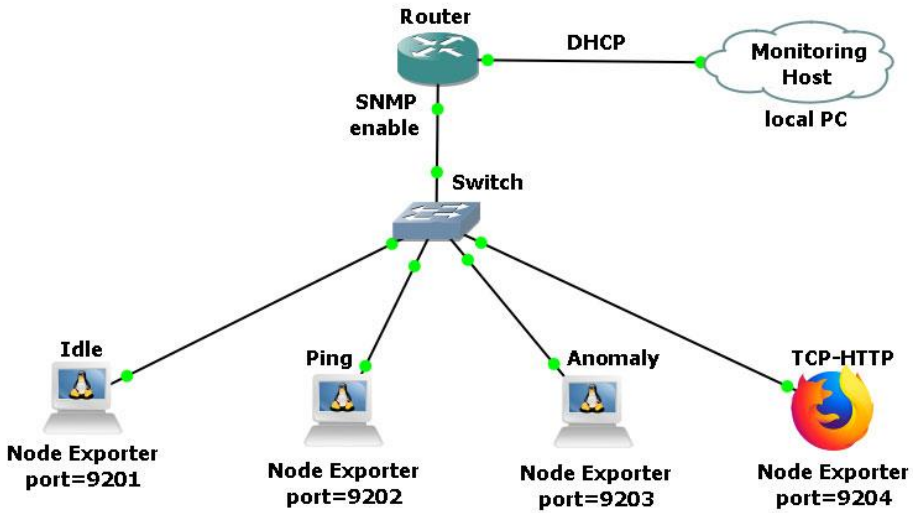
Data Collection & Visualization:
-Prometheus collects relevant metrics, which are visualized in real time using Grafana

AI Models:
- machine learning models that detect anomalies and predict future traffic patterns based on the collected metrics



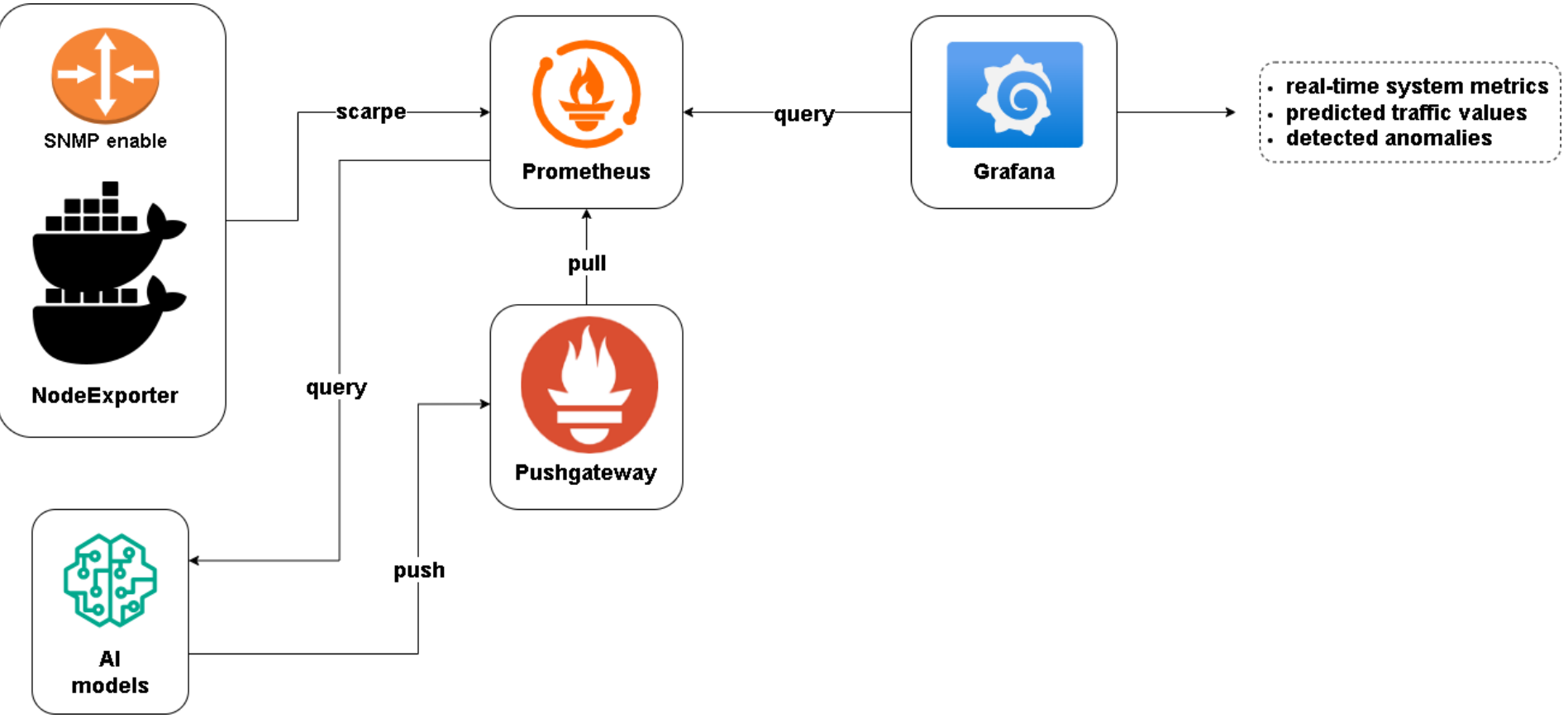
GNS3 VIRTUAL TOPOLOGY

- Docker containers (Alpine-based and Webterm) are connected to a Cisco router
- Each container runs a Node Exporter, which exposes system metrics such as: CPU usage, memory consumption, network I/O (bytes in/out), disk activity
- The Cisco router acts as the central gateway and is connected to the host PC via the GNS3 Cloud node(bridge)
- Port forwarding is configured on the router to allow the local PC to access each container's metrics over unique ports



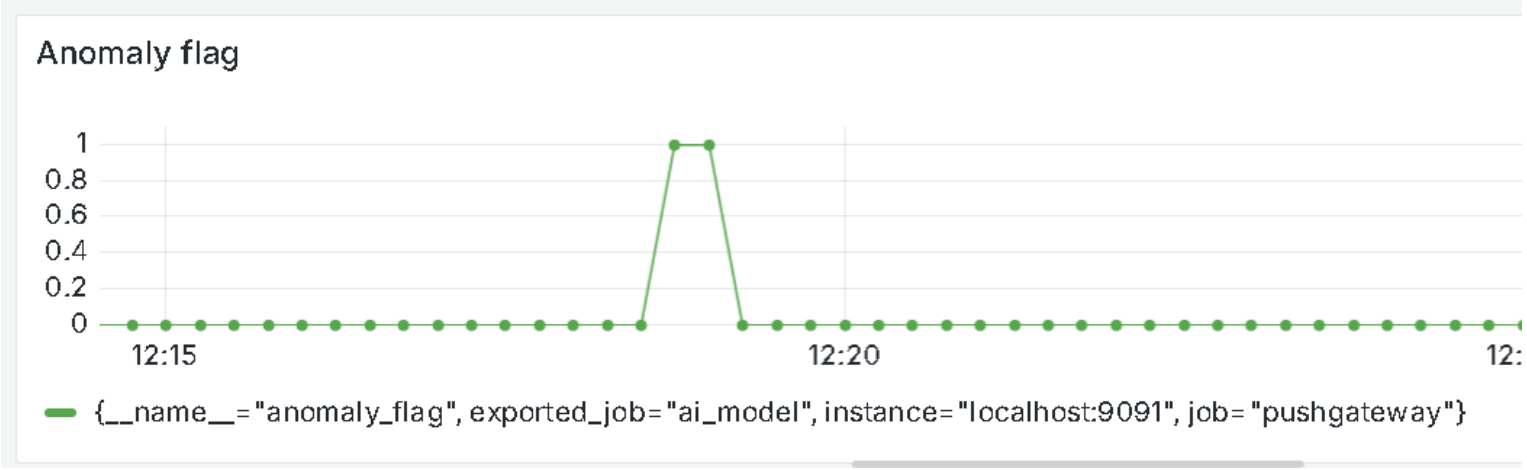
- Traffic is generated from containers to simulate real network activity:
  - idle: no significant traffic
  - ping: sends periodic ICMP requests
  - TCP-HTTP: uses curl/wget to generate HTTP/TCP
  - anomaly: random bursts or scripted heavy load for testing AI detection

METRIC FLOW & PROCESSING



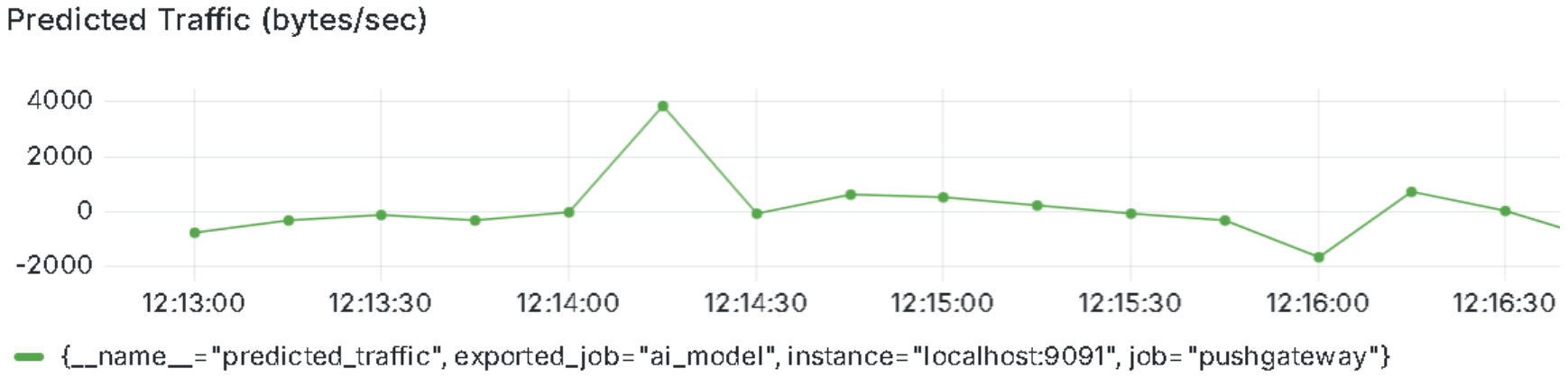
ANOMALY DETECTION (ISOLATION FOREST)

- Model used: Isolation Forest
- Unsupervised algorithm that isolates outliers in metric distributions
- Trained every 60 seconds on the last 10 minutes of data
- Metric used: rate(node_network_receive_bytes_total)
- Output: 0 = normal, 1 = anomaly
- Accuracy observed during test: 87–90%
- Example anomalies: sudden spikes from curl, download bursts



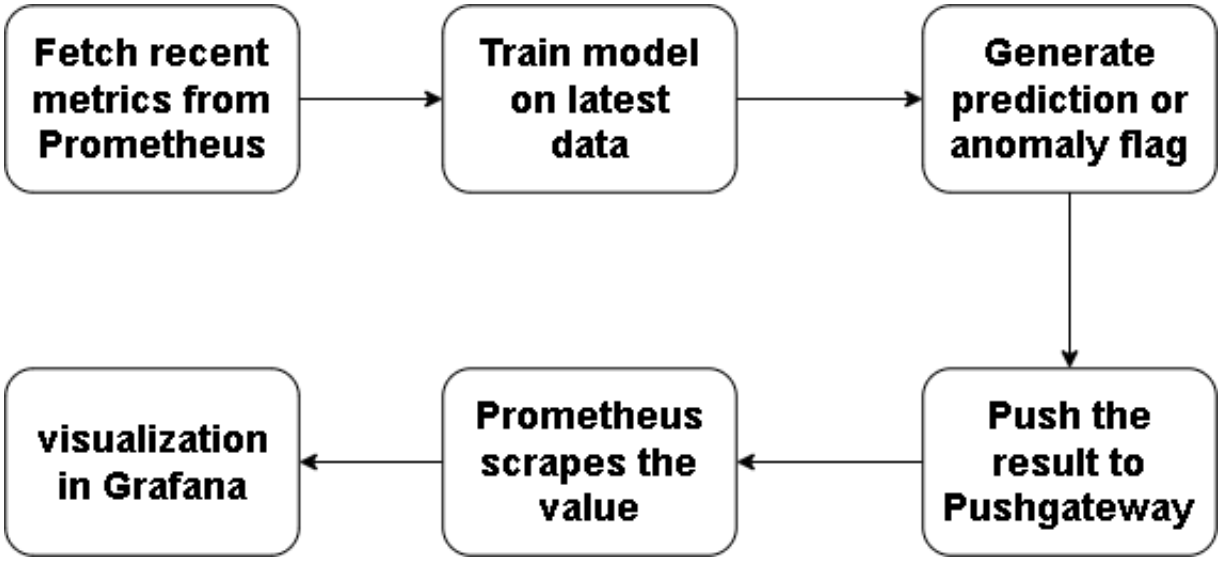
TRAFFIC PREDICTION (LSTM)

- Model used: LSTM (Long Short-Term Memory)
- Designed for time-series forecasting
- Learns patterns from network traffic (bytes/sec over time)
- Windowed training over 10-sample sequences
- Predicts next value and pushes result to Pushgateway
- Accuracy: Mean squared error (MSE) was low (~0.01–0.05) in stable traffic
- Negative outputs indicate model uncertainty or insufficient training data



AI EXECUTION & AUTOMATION

- Python scripts are scheduled to run every 60 seconds using a batch (.bat) script
- Each execution follows this workflow below:



- The process is fully automated and repeats continuously
- No user intervention is required

CONCLUSIONS

- The project demonstrates how AI can enhance traditional monitoring
- Combines:
  - Open-source tools (Prometheus, Grafana, Docker, GNS3)
  - ML models (Isolation Forest, LSTM)
  - Real traffic simulation via containers
- Real-time detection and forecasting is feasible and effective
- Offers flexibility, modularity, and educational value
- Pushgateway enables tight integration between scripts and monitoring tools
- Achieved good prediction reliability and anomaly detection under controlled conditions
- System runs automatically, adapting to recent network data

FUTURE WORK

- Use hybrid models (e.g., Autoencoders + Isolation Forest)
- Save trained models to reduce retraining overhead
- Add more metrics: latency, packet loss, jitter
- Enable feedback loop or alert validation in Grafana
- Expand testbed with multiple routers, subnets, traffic types
- Try supervised ML with labeled attack/normal traffic datasets
- Explore streaming data pipelines (e.g., Kafka + Prometheus)

REFERENCES (SELECTION)

1. A.O. Salau and M.M. Beyene, “Software defined networking based network traffic classification using machine learning techniques,” Scientific Reports, vol. 14, no. 20060, 2024. doi: 10.1038/s41598-024-70983-6)
2. Y. Turk, E. Zeydan, and Z. Bilgin, “A Machine Learning Based Management System for Network Services,” in 2019 Int. Conf. on Wireless and Mobile Computing, Networking and Communications (WiMob), Barcelona, Spain, 2019, pp. 1–8. doi: 10.1109/WiMOB.2019.8923572.
3. B. Lypa, I. Horyn, N. Zagorodna, D. Tymoshchuk, and T. Lechachenko, “Comparison of Feature Extraction Tools for Network Traffic Data,” CEUR Workshop Proceedings, ITTAP 2024, vol. 3373, pp. 486–506.
4. D.J. Fonseca and W.Z. Zheng, “Using GNS3 to enhance the practical learning of computer network design and implementation,” International Journal of Engineering Education, vol. 34, no. 4, pp. 1332–1342, 2018.
5. I.D. Emmanuel, N. Linge, and S. Hill, “Analysis of SD-WAN Packets using Machine Learning Algorithm,” in 2023 Conf. on Information Communications Technology and Society (ICTAS), Durban, South Africa, 2023, pp. 1–6. doi: 10.1109/ICTAS56421.2023.10082743.